Report: Pentium M Hybrid Branch Predictor

What I did

Implemented `pm_predictor` in `my_predictor.h` (`predict.cc` is unchanged).

Structure

- **Bimodal table:** 4096 2-bit saturating counters indexed by the low 12 branch-address bits (x86 instructions are not word-aligned, so no bits are skipped). Initialised weakly not-taken.
- **Global predictor:** 1024 rows x 2 ways. Each entry: valid, 10-bit tag, 2-bit counter, LRU rank. Hash = branch address XOR 16-bit global history register (GHR); upper 10 bits select the row, lower 10 bits are the tag (same split as the worked example in the assignment).
- **Prediction:** global hit → use the global counter; miss → use the bimodal counter.
- **Update:** bimodal is always trained. On a global hit that counter is trained and the way becomes MRU. On a miss a new entry is allocated (invalid way first, otherwise LRU) with a weak counter toward the actual outcome, **only if the bimodal prediction was wrong**. The GHR shifts in every conditional outcome.
- Non-conditional branches are predicted taken and do not touch state. No PIR, no loop predictor, no target prediction.

No external cache code was used; the 2-way set-associative table is written from scratch.

Results (MPKI, direction mispredictions per 1000 instructions, lower is better)

Baseline is the provided 32K-entry gshare.

Trace	Hybrid (mine)	gshare (given)
gzip	12.726	12.473
vpr	14.414	13.415
gcc	13.285	11.254
mcf	18.933	15.837
crafty	7.601	5.837
parser	10.976	10.008
compress	8.065	7.831
jess	1.130	1.562
raytrace	3.065	2.756
db	3.643	3.909

Trace	Hybrid (mine)	gshare (given)
javac	2.367	2.267
mpegaudio	2.377	2.188
mtrt	2.987	2.657
jack	2.421	3.033
eon	1.932	1.807
perlbmk	1.809	2.554
gap	3.931	3.926
vortex	0.787	1.222
bzip2	0.057	0.094
twolf	21.987	21.489
average	**6.725**	**6.306**

The hybrid beats gshare on 6 of 20 traces (jess, db, jack, perlbmk, vortex, bzip2) but loses on the hard integer traces (mcf, gcc, crafty) and on average (6.725 vs 6.306). Its tables are smaller (about 2K tagged entries + 4K bimodal vs 32K counters), and a tagged table needs enough entries not to thrash on branch-heavy programs.

Design-space sweep (average MPKI over all 20 traces)

Variant	Avg MPKI
GHR 12, tag 8 (first version)	7.127
GHR 16, tag 10 (final)	**6.725**
Allocate on every global miss (instead of on bimodal mispredict)	8.723 (with GHR 12/tag 8)
GHR 8	8.717
4096 global rows	7.096
16K bimodal	7.007
Index with word address (shift 2)	7.388
1-way global	7.840
4-way global	6.614

(All rows except the final one are changes relative to the GHR 12 / tag 8 version.)

Takeaways: allocating only when the bimodal predictor is wrong matters most (it keeps easy, bimodal-friendly branches out of the global table); longer history helps; associativity helps (1-way 7.84 → 2-way 7.13 → 4-way 6.61); skipping the low address bits hurts on x86.

What works / limitations

Everything required works and compiles with `make`. Not implemented (per the assignment): loop predictor, PIR/hash between IR and PIR, branch target prediction (so target MPKI is not meaningful). The reported number is direction MPKI as printed by `predict`.